2025年陕西省赛选拔

陕西师范大学

2025年4月20日

题目列表

A	小A爱折跃
B	Locked Calculator
C	你喜欢排列吗?
D	你喜欢运算吗?
E	我要玩原神
F	太阳轰炸
G	我们喜欢字符串
H	一起选西瓜
I	最大上升子序列

本试题册共9题，共10页。若您的试题册不完整，请立即通知志愿者。

Problem A. 小A爱折跃

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

有 n 座城市，分别编号为 $1, 2, \ldots, n$，每座城市持有一台传送到其他城市的折跃门。每台折跃门只能传送到指定编号的城市，且每座城市只能被一台折跃门传送到。给定数组 a，其中 a_i 表示第 i 座城市持有的折跃门可以传送到第 a_i 座城市。

为了加快城市之间的交通速度，市长们委托小 A 为他们进行折跃门的调整，使得从任意一座城市出发，都可以通过折跃门到达任意一座城市。

小 A 可以任意交换两座城市持有的折跃门，他希望知道最少需要交换多少次，才能使得从任意一座城市出发，都可以通过折跃门到达任意一座城市。

Input

第一行一个整数 T $(1 \le T \le 10^4)$，表示测试用例数。

对于每个测试用例：

第一行一个整数 n $(1 \le n \le 10^5)$，表示城市的数量。

第二行共 n 个整数 $a_1, a_2, \ldots, a_n$ $(1 \le a_i \le n)$，表示第 i 座城市持有的折跃门可以传送到第 a_i 座城市。

保证所有测试用例的 n 之和不超过 10^5。

Output

对于每个测试用例，输出一行一个整数，表示最少需要交换多少次，才能使得从任意一座城市出发，都可以通过折跃门到达任意一座城市。

Example

standard input	standard output
5 1 1 2 1 2 3 1 3 2 4 1 3 2 4 5 3 4 5 2 1	0 1 1 2 1

Note

对于第一组测试用例：显然只有一座城市，不需要交换折跃门，因此答案为 0。

对于第三组测试用例：只需交换城市 1 与其它城市中任意一个折跃门即可使任意两座城市互通，因此答案为 1。

Problem B. Locked Calculator

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

一个计算器上有 $0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, \times, \div, =$ 这十五个按键。

现在所有的按键都被锁上了，给你一个非负整数 n ，请问最少需要激活几个按键才能够打出这个数?

被激活的按键可以使用任意次。

注：如果输入的是例如 $1+2$ 的表达式，需要按下 $=$ 才能得到 3；而直接输入 3 不需要按下 $=$。

Input

输入第一行包含一个整数 T $(1 \le T \le 10^3)$ ，代表测试组数。

随后 T 行，每行包含一个非负整数 n $(0 \le n \le 10^9)$ 。

Output

输出 T 行，代表第 i 个测试点最少需要的按键数。

Example

standard input	standard output
3 0 123 1296	1 3 3

Note

对于 0 ，可以直接使用按键 0 打出。

对于 123 ，可以直接使用按键 $1, 2, 3$ 打出。

对于 1296 ，可以使用按键 6 和按键 $\times$ 打出 $6 \times 6 \times 6 \times 6$ ，再使用按键 $=$ 得到 1296 。

Problem C. 你喜欢排列吗?

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

众所周知排列组合是非常非常重要的数学知识。最近小Y想要进一步学习了解排列组合的知识，小 Y 发现她对对排列特别感兴趣。

在这里我们定义长度为 n 的排列是一个整数序列，使得从 0 到 $n-1$ 的每个整数在其中只出现一次。例如，$[0,2,1]$ 是长度为 3 的排列，而 $[0,2,2]$ 和 $[1,2,3]$ 都不是。

对于三个长度都为 n 的排列组成的三元组 (a,b,c)，当且仅当对于任意的 $i(0 \le i \le n-1)$ 满足 $a_i + b_i \equiv c_i \ mod \ n$ 时，这样的三元组被称为幸运排列三元组。符号 a_i 表示排列 a 的第 i 个元素。上述模相等意为 $a_i + b_i$ 除以 n 的余数与 c_i 除以 n 后的余数相等。

现在，她想要找到一个长度为 n 的幸运排列三元组。你能帮帮她吗?

Input

第一行包含一个整数$n(1 \le n \le 10^5)$。

Output

如果不存在长度为 n 的幸运排列三元组，则打印 -1。

否则，输出三行。每行包含 n 个以空格分隔的整数。第一行输出排列 a，第二行输出排列 b，第三行输出排列 c。

如果有多种解决方案，输出任意一种即可。

Examples

standard input	standard output
2	-1
3	0 1 2 2 0 1 2 1 0

Problem D. 你喜欢运算吗?

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

小Y现在正在学习两个正整数之间的一种新的运算，用 $\bigotimes$ 表示。这种运算的结果是通过将两个整数中每个数字的所有多个结果串联起来计算的。

形式上说，第一个整数$A = a_1a_2\ldots a_n$，其中a_i表示A中的第 i 位，第二个整数$B = b_1b_2\ldots b_n$,其中b_i表示B中的第i位。n 为整数 A 的位数，m 为整数 B 的位数。

我们有$A\bigotimes B = \sum_{i=1}^{n}\sum_{j=1}^{m} a_ib_j = a_1b_1 + a_1b_2 + \cdots + a_1b_m + a_2b_1 + \cdots + a_nb_m$

请注意，a_ib_j的结果被认为是 string (字符串)（如果a_ib_j>0，则不带前导零，或者如果a_ib_j>0，则仅包含一个 0），而不是正常整数。此外，这里的 sum 表示 string concatenation (字符串连接)，而不是正常的加法运算。

例如，$23\bigotimes 45 = 8101215$。因为 8=2×4，10=2×5，12=3×4 和 15=3×5。

小Y的脑袋瓜还不错，很快就知道如何做 $\bigotimes$ 的逆运算。现在，她给出了 $\bigotimes$ 运算的结果以及两个原始整数中的位数。请帮助她恢复两个原始整数 A 和 B。

Input

有多个测试用例。输入的第一行包含一个整数 T，表示测试用例的数量。对于每个测试用例:

第一行包含两个正整数 n 和 m $(1 \le n, m \le 2 \times 10^5)$，其中 n 表示 A 的长度，m 表示 B 的长度。这里，整数的长度是指在不带前导零的十进制记数法中写入数字时字符串的长度。

第二行只包含一个不带前导零的正整数 C，表示 $A\bigotimes B$ 的结果。C 的长度不超过2×10^5。

保证所有测试用例的 C 长度总和不会超过 2×10^6。

Output

对于每个测试用例输出一行。

如果存在这样的 A 和 $B, A\bigotimes B = C$，则输出一行，其中包含由一个空格分隔的两个整数 A 与 B。请注意，A 和 B 应该是不带前导零的正整数，A 的长度应该正好是 n，B 的长度应正好是 m。

如果存在多个有效答案，则输出具有最小 A 的答案；如果仍然有一个以上的答案，请输出其中一个最小的 B。

如果不存在这样的 A 和 B，请在单行上打印 Impossible。

Example

standard input	standard output
4 2 2 8101215 3 4 100000001000 2 2 80101215 3 4 1000000010000	23 45 101 1000 Impossible Impossible

Problem E. 我要玩原神

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	256 megabytes

熠熠星辰闪耀天际

旅程中的每一次相逢

都将被细细镌刻在

旅者传颂的歌谣中

Tartarns_yan 准备和 Zoahhhh 一起遨游提瓦特大陆。提瓦特大陆可以视为 n 个点 m 条边的无向连通图，如果 u 和 v 之间存在一条无向边，则从 u 到 v 或者从 v 到 u 需要的时间为 c_{uv} 。

刚开始 Tartarns_yan 位于 1 号节点，Zoahhhh 位于 t 号节点。Tartarns_yan 想要尽快赶到 Zoahhhh 身边，不过 Tartarns_yan 经过的每一个节点都有可能刷新委托。为了帮 Zoahhhh 抽到喜欢的角色，Tartarns_yan 会顺手解决这些委托并获取原石。经过路上的第 i 个点时 Tartarns_yan 需要 w_i 的时间完成委托，最多一共刷新 $n-1$ 个委托。现在 Tartarns_yan 想知道他最短需要多少时间找到 Zoahhhh。

换而言之，一共有 q 次询问，每次询问给出 $t, w_1, w_2, ..., w_{n-1}$ $(t \neq 1)$ ，询问从 1 到 t 所需的最短时间。如果当前的路径经过了 $k+1$ 个节点，分别为 $u_0 = 1, u_1, u_2, .., u_k = t$ ，则 Tartarns_yan 所需额外的时间为 $w_1 + w_2 + ... + w_k$。

Input

第一行给定两个整数 n, m $(1 \leq n \leq 500, 1 \leq m \leq 10^5)$ 。

第二行开始 m 行，每行三个整数 u, v, c_{uv} $(1 \leq c_{uv} \leq 10^9)$。

第 $m+2$ 行给定一个整数 q $(1 \leq q \leq 5000)$ 。

然后开始 q 行，每次询问给出 n 个整数 $t, w_1, w_2, ..., w_{n-1}$ $(2 \leq t \leq n, 0 \leq w_i \leq 10^9)$ 。

Output

q 行，每行一个整数表示答案。

Example

standard input	standard output
4 5 1 2 1 2 3 1 2 4 1 3 4 1 1 4 3 2 4 0 0 0 4 0 2 0	2 3

Problem F. 太阳轰炸

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

背景：阿塔尼斯，达拉姆的大主教，在艾尔又一次沦陷之后指挥着星灵的最后一艘方舟舰：亚顿之矛。作为艾尔星灵数千年来的智慧结晶，亚顿之矛除了搭载了以太阳能碎片为核心的兵工厂之外，还配备了诸如汇聚射线、太阳能射线枪等威力强大的支援武器。而在这些武器中，最负盛名、也最让敌人胆寒的就是太阳轰炸。

太阳轰炸是一件威力巨大的对星球武器。在太阳轰炸开火时，亚顿之矛将聚集太阳能核心中的太阳能量，向目标坐标发射成百上千枚火焰飞弹。虽然这些火焰飞弹精准度较差，但太阳轰炸的高攻击频率仍然可以让地面上的敌人无法躲避，化为灰烬。

在这一次的行动中，阿塔尼斯的目标是一枚臭名昭著的虚空碎片。在俯视视角下，虚空碎片的投影是一个半径为 R_1 的圆，太阳轰炸的攻击散布范围是一个半径为 R_2 的圆。这两个圆的圆心均为原点 $(0,0)$。太阳轰炸将射出 n 枚火焰飞弹，每一枚火焰飞弹等概率地落在攻击散布范围内每一个点上，所有火焰飞弹的落点相互独立。火焰飞弹的伤害范围是以落点为圆心，半径为 r 的圆。若火焰飞弹的伤害范围和虚空碎片的投影相交，则该枚火焰飞弹命中虚空碎片，造成 a 点伤害。若总伤害大于等于 h，则虚空碎片会被摧毁。

摧毁这枚虚空碎片对阿塔尼斯的战略部署非常重要，因此阿塔尼斯想要知道一次太阳轰炸能够摧毁这枚虚空碎片的概率。你需要输出答案对质数 10^9+7 取模的值。

Input

仅一行，包含六个整数 $n, R_1, R_2, r, a, h (1 \le n \le 5\times 10^6, 1 \le R_1, R_2, r \le 10^8, 1 \le a, h \le 10^8)$，含义见题目描述。

Output

一个整数，表示答案。

Example

standard input	standard output
3 2 4 1 1 1	636962896

Note

答案对质数 10^9+7 取模的定义：设 $M=10^9+7$，可以证明答案可表示为一个既约分数 $\frac{p}{q}$ ，其中 p,q 均为整数且 q 模 M 不余 0。按照逆元的定义，输出 $p\cdot q^{M-2}$ 即可。

Problem G. 我们喜欢字符串

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

可怜的Tartarns_yan总是被字符串和回文数击溃, 她决定报复社会。

现在她拿出了一个长度为 n 的字符串 s ，你可以进行一次替换操作，且**必须用和原先不同的字符进行替换。**

换句话说, 你可以进行一次下面的操作（**不能不进行操作**）：

- 选择一个位置 $i(0 \le i < n)$ ，并令 $s_i = x$，其中 x 为小写字母且不能与原来的 s_i 相等。

如果能使用一次操作将其变为回文串, 请输出 YES , 否则输出 NO。

注：回文串是一个正读和反读都一样的字符串。例如， $aaaa$、aba 都是回文串，而 aab、$bbbba$不是。

Input

第一行输入一个正整数 $n(2 \le n \le 10^5)$ ，表示字符串长度。

接下来一行输入一个长度为 n 的字符串 s ，保证字符串中不包含空格。

Output

输出一行一个字符串，表示答案。

Examples

standard input	standard output
5 aabbb	NO
5 abbbb	YES

Note

对于样例1，可以证明无论怎么替换都无法使得 s 为回文串。

对于样例2，可以将 s_4 替换为 a，得到 $abbba$ 是回文串。

Problem H. 一起选西瓜

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

小Y喜欢吃西瓜，于是她和导师制造了一个智能西瓜扫描仪，用来检测一个西瓜的味道如何。这个神奇的扫描仪对于扫描的每个西瓜的图像会输出一个 32 位整数来表示这个西瓜的数据。

她一共扫描了 n 个西瓜，但有时她也会重新扫描一个西瓜，导致这个西瓜的 32 位整数发生更新。

于是她联想起了她学过的异或知识，她觉得异或操作非常有趣，她想要用这些异或知识分析这些西瓜，她每次选取一个区间从 l 至 u，她想要得到这个区间内所有子区间的异或和的异或和。

例如 $l = 2, u = 4$ 的情况，记西瓜数据序列 A 中第 i 个整数是a_i，那么她要求的就是

$$a_2 \oplus a_3 \oplus a_4 \oplus (a_2 \oplus a_3) \oplus (a_3 \oplus a_4) \oplus (a_2 \oplus a_3 \oplus a_4)$$

小Y相信你已经早早知道异或的知识，因此她在这里就不多说异或的知识了。请你来解决这个问题吧！

Input

第一行输入两个正整数 n, q，表示西瓜数量和操作次数。

接下来一行 n个非负整数，表示每个西瓜扫描得到的数值 ，从 1 开始编号。

接下来 q 行，每行三个数:

- 如果第一个数是 1，接下来输入一个正整数 i 与非负整数 j，表示将第 i 个西瓜的扫描值a_i 修改为 j。
- 如果第一个数是 2，接下来输入两个正整数 u, l 表示询问这个区间的答案。

Output

对于每组询问，输出一行一个非负整数，表示所求的总异或和。

Example

standard input	standard output
3 3 1 2 3 2 1 3 1 1 3 2 1 3	2 0

Note

$0 \le a_i \le 10^9$

$1 \le n, q \le 2 \times 10^5$

Problem I. 最大上升子序列

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

给你两个 n×m矩阵，每个矩阵的元素从 1到 n×m不等，且成对不同。你需要找出这两个矩阵之间大小最大的公共子矩阵

例子:

设矩阵 A:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 8 & 7 & 9 \end{pmatrix}$$

矩阵 B:

$$\begin{pmatrix} 5 & 6 & 1 \\ 7 & 9 & 3 \\ 2 & 4 & 8 \end{pmatrix}$$

那么它们的最大公有子矩阵为:

$$\begin{pmatrix} 5 & 6 \\ 7 & 9 \end{pmatrix}$$

Input

第一行输入包含两个整数 n $(1 \le n \le 1000)$ 和 m $(1 \le m \le 1000)$, 分别表示每个矩阵的行数和列数。

接下来的 n 行, 每行包含 m 个数字, 表示第一个矩阵 $A = (a_{i,j})_{n\times m}$. 同样, 接下来的 n 行每行包含 m 个整数，表示第二个矩阵 $B = (b_{i,j})_{n\times m}$.

可以保证 $1 \le a_{i,j}, b_{i,j} \le n \times m$, 和 $a_{i_1,j_1} \ne a_{i_2,j_2} \wedge b_{i_1,j_1} \ne b_{i_2,j_2}$ 在每一对 (i_1, j_1) 和 (i_2, j_2), 其中 $i_1 \ne i_2 \vee j_1 \ne j_2$总是成立.

Output

输出一个整数，代表最大公共子矩阵的大小。

Example

standard input	standard output
3 4 5 6 7 8 1 2 3 4 9 10 11 12 5 6 8 7 1 2 4 3 12 11 10 9	4